Monster Jump

Projektdokumentation

Informatik 3 · SoSe 2026

Dozentin: Aysun Cetinalp

Name	Matr. Nr.
Tobias Hahn	817616
Lutz Morath	818145
Jonas Uzelac	804756

Gruppenname: Gruppe B
Abgabedatum: 16.06.2026

Inhaltsverzeichnis

Inhaltsverzeichnis **2**

1 Projektbeschreibung **3**

2 Spielreferenzen und Inspiration **3**
2.1 Fireboy and Watergirl . . . 3
2.2 Super Mario Bros . . . 3

3 Herausforderungen und Lösungsdesign **4**
3.1 Grafische Oberfläche . . . 4
3.2 Objektplatzierung . . . 4
3.3 Inputhandling . . . 4
3.4 Kollisionsbehandlung . . . 4
3.5 Multiplayer . . . 4
3.6 Animation . . . 4
3.7 Verschiedene Menüs . . . 4

4 Zeitplan und Aufgabenverteilung **5**

5 UML-Klassendiagramm **6**

6 API-Dokumentation **7**
6.1 pygame . . . 7
6.2 pytmx . . . 7
6.3 socket . . . 7
6.4 json . . . 7
6.5 struct . . . 7
6.6 unittest & pytest . . . 7

7 Quellcode-Metriken **8**

8 Retrospektive **8**
8.1 Was hat gut funktioniert? . . . 8
8.2 Was würden wir beim nächsten Mal anders machen? . . . 8

1 Projektbeschreibung

Genre: Koop 2D Platformer

Technologie/Engine: Python + PyGame

Kurzbeschreibung: Monster Jump ist ein 2D Platformer, bei dem zwei Spieler gemeinsam an einer Tastatur oder im Multiplayer gemeinsam spielen. Ziel ist es, kooperativ einen Ausweg aus dem Level zu finden, indem Rätsel gelöst und Hindernisse überwunden werden.

2 Spielreferenzen und Inspiration

2.1 Fireboy and Watergirl

- **Genre / Plattform:** Koop 2D Platformer
- **Was wurde übernommen / inspiriert:**
  Grundspielprinzip, Level Aufbau, Spieldynamik

2.2 Super Mario Bros

- **Genre / Plattform:** 2D Platformer
- **Was wurde übernommen / inspiriert:**
  Movement, Grafik

3 Herausforderungen und Lösungsdesign

3.1 Grafische Oberfläche

Um überhaupt ein Window zeichnen zu können und Objekte grafisch darzustellen haben wir uns dazu entschieden die Bibliothek PyGame zu verwenden, da diese perfekt zu unserem Anwendungsgebiet passt.

3.2 Objektplatzierung

Um Objekte nicht explizit mittels Koordinaten im Code platzieren zu müssen, nutzen wir den Editor Tiled und die Bibliothek Pytmx. So kann man im Editor die Objekte grafisch platzieren und mit der entstandenen tmx Datei sowohl Objekte als auch Grafiken platzieren bzw. darstellen.

3.3 Inputhandling

Um eine gute seperation of concerns einzuhalten, wird das komplette Inputhandling in eine dedizierte InputHandler Klasse ausgelagert. Dort wird der Spieler Input sowie der Input für für die Menüs verarbeitet.

3.4 Kollisionsbehandlung

Da PyGame auch für diese Probleme Lösungen anbietet, verwenden wir die eingebaute Kollisionsbehandlung. Mit der Klasse pygame.Rect lassen sich Kollisionen leicht überprüfen und weiterverarbeiten

3.5 Multiplayer

Hier haben wir uns dazu entschieden, ein Server Client System umzusetzen. Dafür benutzen wir die Bibliothek socket, um einen TCP Server aufzusetzen. Ein Spieler ist der Host und startet den Server, der andere muss über die lokale IP Adresse im gleichen Netzwerk beitreten.

3.6 Animation

Für die Animation des Charakters gibt es je nach Bewegungsstatus verschiedene Bilddateien, durch die Frameweise iteriert wird. So entsteht eine flüssige Animation des Charakters.

3.7 Verschiedene Menüs

Um zwischen allen Menüs und Level zu wechseln, nutzen wir einen Game State Manager, welcher alle Zustände kennt und zwischen ihnen wechselt. Je nach Zustand wird eine andere grafische Oberfläche dargestellt.

4 Zeitplan und Aufgabenverteilung

Zeitplan	Datum	Verantwortlich	Status
Projektkonzept & Anforderungen	20.04.2026	Gemeinsam	✓ Erledigt
Kontakt zu Prof. Dr. Kücherer (EA)	20.04.2026	Gemeinsam	✓ Erledigt
UML-Klassendiagramm (Entwurf)	20.04.2026	Gemeinsam	✓ Erledigt
Fenster Aufbau	27.04.2026	Lutz Morath	✓ Erledigt
Spieler Physik, Input, Bewegung	27.04.2026	Tobias Hahn	✓ Erledigt
Prototyp: Level	04.05.2026	Jonas Uzelac	✓ Erledigt
Kollisionsbehandlung	04.05.2026	Tobias Hahn	✓ Erledigt
Zweiter Spieler lokal	11.05.2026	Jonas Uzelac	✓ Erledigt
Level Mechanik (Start, Ziel, Menu)	11.05.2026	Lutz Morath	✓ Erledigt
Tests	11.05.2026	Gemeinsam	✓ Erledigt
Prototyp / spielbare Demo	11.05.2026	Gemeinsam	✓ Erledigt
Modellierung Sparx EA fertig	01.06.2026	Gemeinsam	✓ Erledigt
Server Client System	20.05.2026	Tobias Hahn	✓ Erledigt
Erweiterte Spielmechaniken	02.06.2026	Jonas Uzelac	✓ Erledigt
Tests	02.06.2026	Gemeinsam	✓ Erledigt
Charakterdesign (Grafik, Animation)	02.06.2026	Lutz Morath	✓ Erledigt
Leveldesign (Grafik, Sound)	02.06.2026	Lutz Morath	✓ Erledigt
Levelaufbau	02.06.2026	Jonas Uzelac	✓ Erledigt
Implementierung abgeschlossen	02.06.2026	Gemeinsam	✓ Erledigt
Dokumentation abgeschlossen	09.06.2026	Gemeinsam	✓ Erledigt
Abgabe	16.06.2026	Gemeinsam	✓ Erledigt

Tabelle 1: Zeitplan

5 UML-Klassendiagramm

Erstellt mit Sparx Enterprise Architect

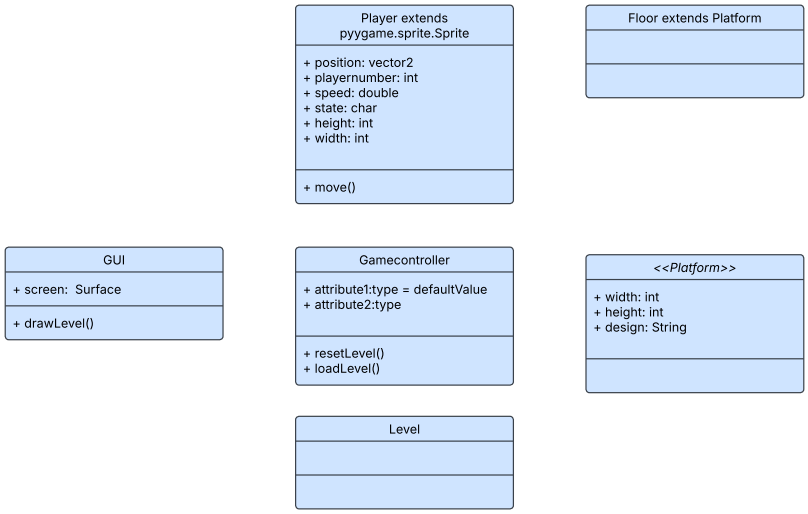


Abbildung 1: UML Klassendiagramm dieses Projekts

6 API-Dokumentation

6.1 pygame

Pygame ist die zentrale API des Projekts und wird für nahezu alle Spielmechaniken verwendet. Über pygame.display hat man Zugriff auf das Window und kann über die blit() Methode alle erstellten Elemente zeichnen.

Darüber hinaus haben alle Spielobjekte von pygame.sprite.Sprite geerbt, um die Kollisionsbehandlung und Grafik der Objekte zu vereinfachen.

Die Tastatureingabe wurde ebenfalls mithilfe von pygame umgesetzt. Über pygame.event.get() bekommt man alle aktuellen Events, darunter auch Tastatureingaben. Um feste FPS zu haben, nutzen wir pygame.time.

6.2 pytmx

Pytmx wird zum Laden von Maps verwendet, die mit dem Tiled Map Editor erstellt wurden. Man kann im Editor mithilfe von Tilesets die Objekte platzieren und erhält eine .tmx Datei. Diese kann dann in Python mithilfe dieser API ausgewertet werden.

6.3 socket

Die API socket wir für die TCP-Netzwerkkommunikation für den Multiplayer-Modus verwendet. Es wird verwendet, um den Server zu erstellen und mit 2 Clients zu verbinden. Die Verbindung funktioniert über die lokale IP Adresse in einem lokalen Netzwerk.

6.4 json

Die json API nutzen wir für die Serialisierung der Daten, welche über den Server versendet werden. Mithilfe von json.dumps werden Daten serialisiert und mit loads deserialisiert. Die Daten werden dabei immer als Dictionaries versendet.

6.5 struct

Wir verwenden struct für die Paketlänge der versendeten Daten. Jedes Paket beinhaltet einen Head mit einer 4-Byte Nachricht, die die Länge des jeweiligen Pakets beschreibt. Ansonsten wüsste man bei der TCP Verbindung nicht, wie lang die Pakete sind und wo eine neue Nachricht beginnt.

6.6 unittest & pytest

Diese beiden APIs werden für unsere Unit Tests verwendet. Sie helfen dabei, indem man über @pytest.fixture kann man vereinfachte Objekte erstellen, die als Parameter an Methoden übergeben werden.

Über unittest.mock kann man Mock Objekte erstellen, die normale Objekte wie unseren Server simulieren, ohne diese wirklich verwenden zu müssen.

7 Quellcode-Metriken

Gemessen zum Zeitpunkt der Abgabe.

Metrik	Wert
Anzahl Dateien (#Files)	326
Lines of Code (LOC)	1573
Anzahl Klassen	21
Anzahl Methoden	135

Tabelle 2: Quellcode-Metriken (ermittelt mit lizard)

8 Retrospektive

8.1 Was hat gut funktioniert?

Die Einbindung des Tiled Map Editors hat gut funktioniert. Die Level sind jetzt einfach zu gestalten und das Spiel ist somit leicht erweiterbar.

Die Umsetzung der Charakter Animation erwies sich als aufwändiger als gedacht. Das Endprodukt ist jedoch zufriedenstellend und die Bilddateien für die Animationen sind leicht austauschbar, wodurch das Aussehen des Spiels einfach verändert werden kann.

Die Spielzustände haben den Arbeitsfluss verbessert. Dadurch war es leicht Menüs zu erweitern oder neue zu erstellen und neue Level schnell einzubinden. Außerdem wurde dadurch der Umfang des Gameloops erheblich reduziert.

8.2 Was würden wir beim nächsten Mal anders machen?

Obwohl wir mit dem Multiplayer zufrieden sind, gibt es noch Funktionen, die beim Multiplayer nicht funktionieren. Das Einbinden von beweglichen Objekten in den Multiplayer funktionierte bis zuletzt nicht. Nächstes mal würden wir bei der Planung der Architektur zu Beginn schon das Server CLient System berücksichtigen.

Außerdem haben wir den zeitlichen Aufwand der Gestaltung unterschätzt. Daher haben wir unseren Spiel Titel und das Aussehen des Charakters und des Levels geändert, da wir keine passenden Assets für unser geplantes Thema gefunden haben.